Computación

Clase 11

Temas

Arreglos bidimensionales : matrices

123.4	-0.098	334.5	1
456.89	22.3	0.0009	34.7
-9.99	100	12.67	2.3333

Si se desea referenciar cada elemento simple de la matriz debemos indicar su posición usando dos subíndices (mediante la fila y columna donde se halla, en forma similar a las matrices algebraicas).

Matriz A

$$
\begin{array}{c}
\\ \text{Fila 1} \\ \text{Fila 2} \\ \text{Fila 3} \\ \text{Fila 4}
\end{array}
\begin{array}{c}
\begin{array}{ccc} \text{Columna 1} & \text{Columna 2} & \text{Columna 3} \end{array} \\
\begin{bmatrix} 12 & 1002 & -9.9 \\ 1.2 & 76.7 & 9.9 \\ 34.09 & 112 & 34.1 \\ 15 & 1 & -0.92 \end{bmatrix}
\end{array}
$$

A_{23}

A_{42}

Matrices: propiedades

- Todas las componentes son del mismo tipo (homogéneo)
- Poseen **dos dimensiones** (fila y columna)
- Cada una de las **dimensiones** tienen límites inferior y superior. Estos límites determinan la extensión de los valores que son usados como subíndices para el acceso.
- El **tamaño** se indica cuando se define y queda invariable a partir de ese momento (del mismo modo que en los arreglos unidimensionales).

.

Arreglos bidimensionales

Estructura donde almacenamos los datos simples, indicando su posición con fila/columna

componentes

$$\begin{matrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \end{matrix}$$

$$\begin{bmatrix} 12 & 1002 & -9.9 \\ 1.2 & 76.7 & 9.9 \\ 34.09 & 112 & 34.1 \\ 15 & 1 & -0.92 \end{bmatrix}$$

Los componentes se asignan a cada posición de memoria [i,j]

Declaración de una matriz: ejemplo

- La matriz se estructura como **un vector de vectores** y podría definirse así aunque no es recomendable pues

Extensión de la 1era dimensión: cantidad de filas

Extensión de la 2da dimensión: cantidad de columnas

TYPE

TMATRIZ = array[1..4] of array[1..3] of real;

arreglo lineal de vectores

arreglo lineal de reales

Declaración de una matriz: ejemplo

Resulta conveniente usar esta notación, mas compacta.

Extensión de la 1era dimensión: cantidad de filas

Extensión de la 2da dimensión: cantidad de columnas

TYPE

TMATRIZ = array[1..4,1..3] of real;

Identificador de tipo

Tipo indice

Tipo base

Ejemplo

```
TYPE
 TMAT= array[1..4,1..3] of real;
 TM = array[-2..2,0..7] of integer;
Var
 A:TMAT;
 B:TM;
 i,j:integer;
Begin
 A[2,1]:=1.2;
 A[3,3]:=34.1;
 i:=1;
 j:=1;
 A[i,j]:=12;
 A[i+2,j+1]:=B[-2,0]*B[-1,2]/5;
 ................
```

Lectura de una matriz : ejemplo

```
Type
 TM=array[1..10,1..10] of real;
Procedure LeeXFilas(Var A:TM; Var N,M:byte);
 Var
  i,j:byte;
 Begin
  Write('Ingrese cantidad de filas <=10');
  Readln(N);
  Write('Ingrese cantidad de columnas <=10');
  Readln(M);
  For i:=1 to N do {fila i-esima}
   For j:=1 to M do
    Begin
     Write('A(',i,',',j,')=');
     Readln(A[i,j])
    End;
 End;
```

Lectura de una matriz

En la memoria se almacenan los datos en forma contigua. Por lo tanto, en el caso de un arreglo bi-dimensional se almacenan las filas contiguas (1ra fila, 2da fila, etc.).
Sin embargo, se puede decidir el orden en el que ingresarán y tratarán los datos: ***por filas*** *o* ***por columnas****.*

Escritura de una matriz en formato matricial

```
Procedure EscribeMat( A:TM; N,M:byte);
Var
 i,j:byte;
Begin
 For i:=1 to N do
  Begin
   For j:=1 to M do
    write(A[i,j]);
   writeln;
  End;
End;
```

Cálculo del máximo y mínimo de una matriz

```
Procedure MaxMin(A:TM; N, M:byte; Var Max, Min:integer);
Var
 i, j:byte;
Begin
 Max:=A[1,1];
 Min:=A[1,1];
 for i:=1 to N do
  for j:=1 to M do
   Begin
    if A[i,j]>Max then
      Max:=A[i,j];
    if A[i,j]<Min then
      Min:=A[i,j];
   End;
End;
```

Ejemplo

Detectar si una matriz es triangular inferior.
Se presentan tres opciones de algoritmos, considerando que el último optimiza el tiempo de ejecución (en número de acciones elementales)

Opción 1

```
Function TriangularInferior1(A:TMatri; N:byte):Boolean;
var
 i, j: byte;
 triangular: boolean;
Begin
 triangular:=true;
 for i:=1 to N do
  for j:=1 to N do
    if i<j then
      if a[i,j]<>0 then
        triangular:=false;
  TriangularInferior1 :=triangular;
End;
```

Opción 2

```
Function TriangularInferior2(A:TMatri; N:byte):Boolean;
var
 i, j: byte;
 triangular: boolean;
Begin
 triangular:=true;
 for i:= 1 to N-1 do
  for j:= i+1 to N do
   if a[i,j]<>0 then
    triangular:=false ;
  TriangularInferior2 :=triangular;
End;
```

Opción 3

```
Function TriangularInferior3(A:TMatri; N:byte):Boolean;
var
 i, j: byte;
 triangular: boolean;
Begin
 i:=1;
 triangular := true;
 while (i <N) and triangular do
  begin
   j:=i+1;
   while (j <= N) and triangular do
    begin
     if a[i,j]<>0 then
      triangular:=false;
     j:=j+1;
    end;
   i:=i+1;
  end;
 TriangularInferior3 :=triangular;
End;
```

Arreglos n-dimensionales

Se podrían especificar n subíndices. En este caso se dice que el arreglo es n-dimensional, y cada componente se denota mediante n índices.

Por ejemplo Z[i,j,k] refiere a un elemento de un arreglo tri-dimensional (pueden pensarse como grupos de matrices).